

13.4-AI-ASS

2303A52008

K.Keerthan

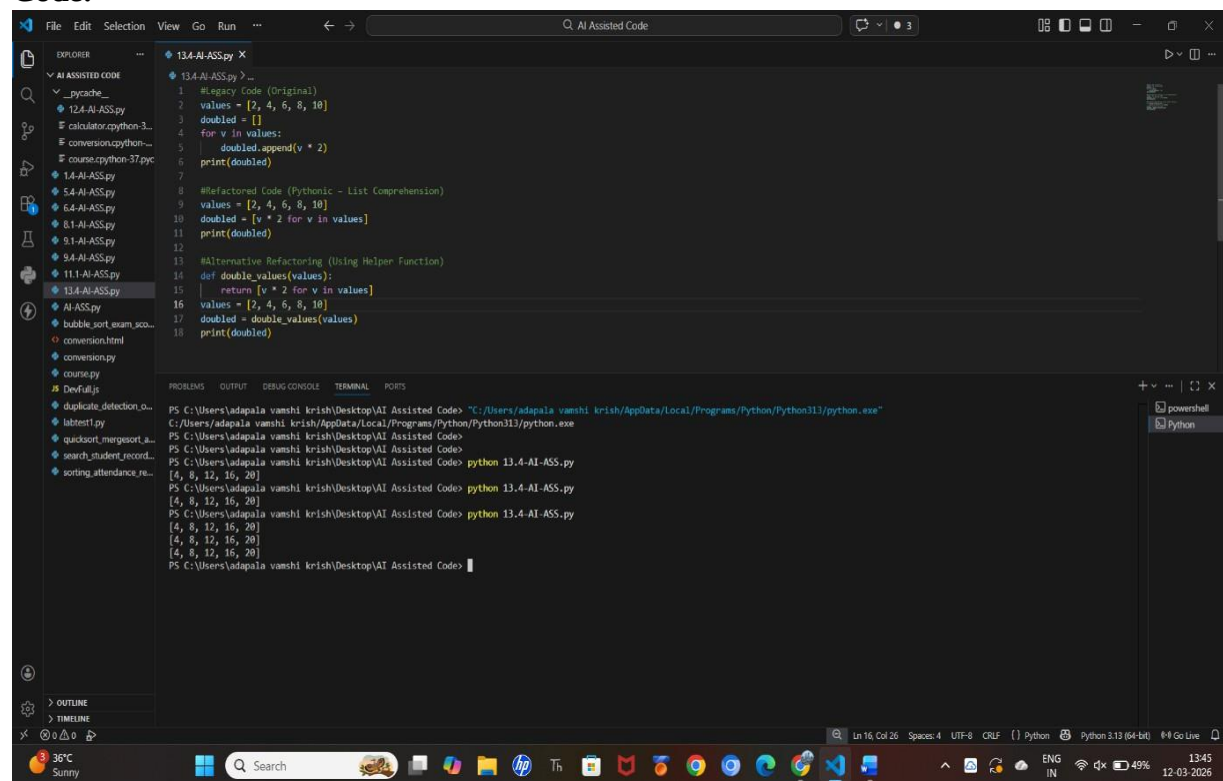
TASK-1

Scenario: (Refactoring Data Transformation Logic)

You are maintaining a data preprocessing script where numerical transformations are written using verbose loops.

Prompt: Refactor the following Python code to make it more Pythonic and readable using list comprehensions while keeping the same output.

Code:



The screenshot shows a Visual Studio Code editor with a file named '13.4-AI-ASS.py'. The code is divided into three sections: 'Legacy Code (Original)', 'Refactored Code (Pythonic - List Comprehension)', and 'Alternative Refactoring (Using Helper Function)'. The 'Legacy Code' uses a for loop to double each value in a list. The 'Refactored Code' uses a list comprehension to achieve the same result in a single line. The 'Alternative Refactoring' uses a helper function to double the values. The terminal at the bottom shows the execution of the code, which outputs the doubled list: [4, 8, 12, 16, 20].

```
1 # Legacy Code (Original)
2 values = [2, 4, 6, 8, 10]
3 doubled = []
4 for v in values:
5     doubled.append(v * 2)
6 print(doubled)
7
8 # Refactored Code (Pythonic - List Comprehension)
9 values = [2, 4, 6, 8, 10]
10 doubled = [v * 2 for v in values]
11 print(doubled)
12
13 # Alternative Refactoring (Using Helper Function)
14 def double_values(values):
15     return [v * 2 for v in values]
16 values = [2, 4, 6, 8, 10]
17 doubled = double_values(values)
18 print(doubled)
```

Terminal Output:

```
PS C:\Users\adapala vamshi krish\Desktop\AI Assisted Code> "C:\Users\adapala vamshi krish\AppData\Local\Programs\Python\Python311\python.exe"
C:\Users\adapala vamshi krish\AppData\Local\Programs\Python\Python311\python.exe
PS C:\Users\adapala vamshi krish\Desktop\AI Assisted Code>
PS C:\Users\adapala vamshi krish\Desktop\AI Assisted Code> python 13.4-AI-ASS.py
[4, 8, 12, 16, 20]
PS C:\Users\adapala vamshi krish\Desktop\AI Assisted Code> python 13.4-AI-ASS.py
[4, 8, 12, 16, 20]
PS C:\Users\adapala vamshi krish\Desktop\AI Assisted Code> python 13.4-AI-ASS.py
[4, 8, 12, 16, 20]
PS C:\Users\adapala vamshi krish\Desktop\AI Assisted Code> python 13.4-AI-ASS.py
[4, 8, 12, 16, 20]
PS C:\Users\adapala vamshi krish\Desktop\AI Assisted Code>
```

Observation:

The AI suggested using list comprehension to replace the explicit loop.

This reduced the number of lines, improved readability, and made the code more Pythonic while maintaining the same output.

TASK-2

Scenario: (Improving Text Processing Code Readability)

You are working on a log message generator that builds messages word by word.

Prompt:

Improve the following Python code that constructs a sentence using repeated string concatenation. Suggest a more efficient and readable approach.

Code:

```
12
13 # Alternative Refactoring (Using Helper Function)
14 # def double_values(values):
15 #     return [v * 2 for v in values]
16 # values = [2, 4, 6, 8, 10]
17 # doubled = double_values(values)
18 # print(doubled)
19
20
21 # Legacy Code (Original)
22 words = ["Refactoring", "with", "AI", "Improves", "quality"]
23 message = ""
24 for w in words:
25     message = w + " "
26 print(message.strip())
27
28 # Refactored Code (Pythonic Approach)
29 words = ["Refactoring", "with", "AI", "Improves", "quality"]
30 message = " ".join(words)
31 print(message)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
[4, 8, 12, 16, 20]
[4, 8, 12, 16, 20]
PS C:\Users\adapala vamsi krish\Desktop\AI Assisted Code> python 13.4-AI-ASS.py
Refactoring with AI Improves quality
Refactoring with AI Improves quality
Refactoring with AI Improves quality
PS C:\Users\adapala vamsi krish\Desktop\AI Assisted Code>
```

Observation:

The AI recommended using the `join()` method to combine the words into a sentence.

This approach avoids repeated string concatenation, improves performance, and makes the code more concise and readable.

TASK-3

Scenario: (Safer Access to Configuration Data)

You are maintaining a configuration loader where dictionary keys may or may not exist depending on deployment settings.

Prompt:

Refactor the following Python code to safely access dictionary keys using a more Pythonic method while preserving the behavior for missing keys.

Code:

```
33
34 # Legacy Code (Original)
35 config = {"host": "localhost", "port": 8080}
36
37 if "timeout" in config:
38     print(config["timeout"])
39 else:
40     print("Default timeout used")
41
42
43 # Refactored Code (Using dict.get() Method)
44 config = {"host": "localhost", "port": 8080}
45 timeout = config.get("timeout")
46 if timeout:
47     print(timeout)
48 else:
49     print("Default timeout used")
50
51
52 # More Pythonic Refactoring
53 config = {"host": "localhost", "port": 8080}
54 print(config.get("timeout", "Default timeout used"))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
[4, 8, 12, 16, 20]
PS C:\Users\adapala vamsi krish\Desktop\AI Assisted Code> python 13.4-AI-ASS.py
Refactoring with AI Improves quality
Refactoring with AI Improves quality
Default timeout used
Default timeout used
Default timeout used
PS C:\Users\adapala vamsi krish\Desktop\AI Assisted Code>
```

Observation:

The AI suggested using the dictionary `get()` method to safely retrieve values.

This simplifies the code and prevents errors when a key is missing while keeping the same behavior.

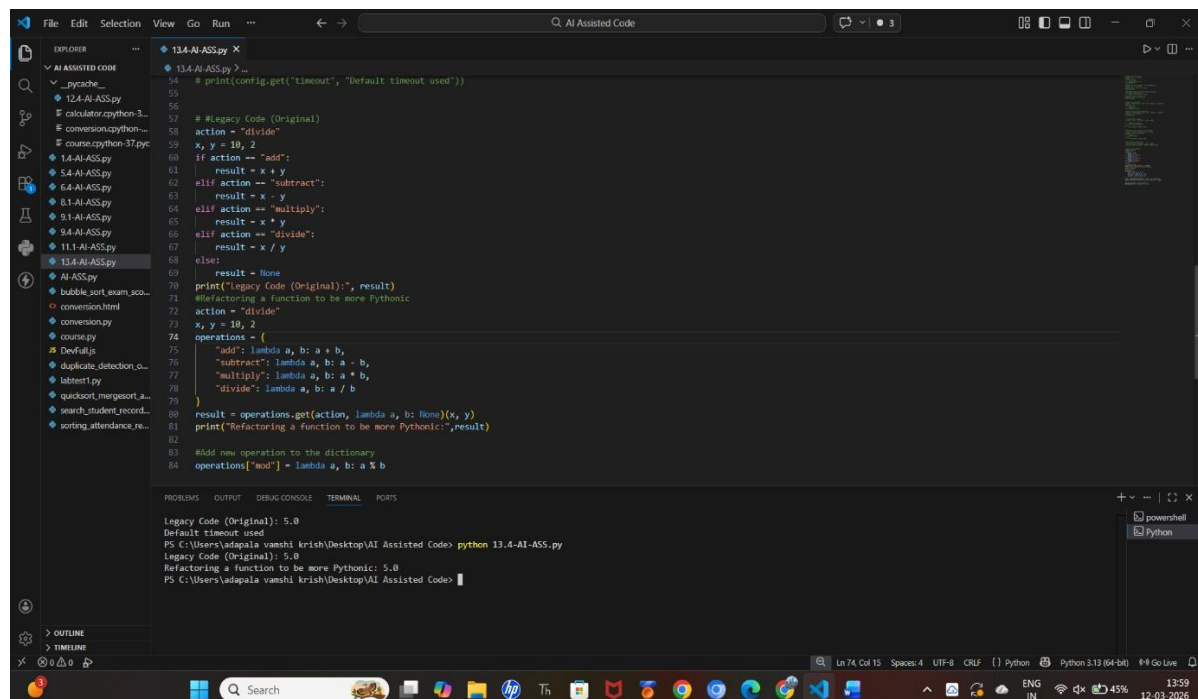
TASK-4

Scenario: (Refactoring Conditional Logic for Scalability)

You are enhancing a utility module that performs different operations based on user input.

Prompt: Refactor the following Python code that uses multiple if-elif conditions. Suggest a cleaner and scalable approach using mapping techniques.

Code:



```
54 # print(config.get("timeout", "Default timeout used"))
55
56
57 # Legacy Code (Original)
58 action = "divide"
59 x, y = 10, 2
60 if action == "add":
61     result = x + y
62 elif action == "subtract":
63     result = x - y
64 elif action == "multiply":
65     result = x * y
66 elif action == "divide":
67     result = x / y
68 else:
69     result = None
70 print("Legacy Code (Original):", result)
71 #Refactoring a function to be more Pythonic
72 action = "divide"
73 x, y = 10, 2
74 operations = {
75     "add": lambda a, b: a + b,
76     "subtract": lambda a, b: a - b,
77     "multiply": lambda a, b: a * b,
78     "divide": lambda a, b: a / b
79 }
80 result = operations.get(action, lambda a, b: None)(x, y)
81 print("Refactoring a function to be more Pythonic:", result)
82
83 #Add new operation to the dictionary
84 operations["mod"] = lambda a, b: a % b
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Legacy Code (Original): 5.0
Default timeout used
PS C:\Users\adapaia\vamshi.krish\Desktop\AI Assisted Code> python 13.4-AI-ASS.py
Legacy Code (Original): 5.0
Refactoring a function to be more Pythonic: 5.0
PS C:\Users\adapaia\vamshi.krish\Desktop\AI Assisted Code> |

Observation:

The AI recommended replacing the multiple **if-elif conditions** with a **dictionary mapping of operations**.

This makes the code easier to maintain, scalable, and cleaner when adding new operations.

TASK-5

Scenario: (Simplifying Search Logic in Collections)

You are reviewing legacy inventory-check code that uses manual loops to find elements.

Prompt: Refactor the following Python code that searches for an item in a list using a loop. Suggest a more concise and readable Pythonic approach.

Code:

The screenshot shows a Visual Studio Code editor window with a file explorer on the left and a terminal at the bottom. The main editor area displays a Python file named `13.4-AI-ASS.py`. The code is divided into three sections: Legacy Code (Original), Refactored Code (Using 'in' Keyword), and Even Short Version. The Legacy Code uses a loop to check if an item is in a list. The Refactored Code uses the `in` operator to check if an item is in a list. The Even Short Version uses the `in` operator to check if an item is in a list. The terminal at the bottom shows the output of running the script, which prints the results for each version.

```
80 # result = operations.get(operation, lambda a, b: None)(x, y)
81 # print("Refactoring a function to be more Pythonic:", result)
82
83 # Add new operation to the dictionary
84 # operations["mod"] = lambda a, b: a % b
85
86
87 #Legacy Code (Original)
88 inventory = ["pen", "notebook", "eraser", "marker"]
89 found = False
90 for item in inventory:
91     if item == "eraser":
92         found = True
93         break
94 print("Legacy Code (Original):", "Item Available" if found else "Item Not Available")
95
96 #Refactored Code (Using 'in' Keyword)
97 inventory = ["pen", "notebook", "eraser", "marker"]
98 found = "eraser" in inventory
99 print("Refactored Code (Using 'in' Keyword):", "Item Available" if found else "Item Not Available")
100
101 #Even Short Version
102 inventory = ["pen", "notebook", "eraser", "marker"]
103
104 print("Even Short Version:", "Item Available" if "eraser" in inventory else "Item Not Available")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\adapala vamshi.krish\Desktop\AI Assisted Code> python 13.4-AI-ASS.py
Legacy Code (Original): Item Available
Refactored Code (Using 'in' Keyword): Item Available
Even Short Version: Item Available
PS C:\Users\adapala vamshi.krish\Desktop\AI Assisted Code>
```

Observation:

The AI suggested using the `in` operator to check if the item exists in the list. This eliminates the manual loop, simplifies the logic, and improves readability.